Modul Aktivitas Praktikum Algoritma dan Struktur Data

Penyusun: Auzi Asfarian, S.Komp., M.Kom

Daftar Isi

Daftar Isi	1
Topik 8: Pengurutan (Sorting)	**2**
Masalah Pengurutan	2
Algoritma Pengurutan	2
Pengurutan pada Python	4
Implementasi Algoritma Bubble Sort	5
Implementasi Algoritma Selection Sort	6
Implementasi Algoritma Insertion Sort	7
Implementasi Algoritma Shell Sort	8
Implementasi Algoritma Merge Sort	10
Implementasi Algoritma Quick Sort	13
Latihan Soal Pengurutan	15
Topik 9: Pencarian (Searching)	**16**
Masalah Pencarian	16
Algoritma Pencarian	16
Pencarian pada Struktur Data di Python	16
Implementasi Algoritma Sequential Search	19
Implementasi Algoritma Binary Search	19
Latihan Soal Pencarian	21

Topik 8: Pengurutan (Sorting)

Aktivitas:
Ikutilah tutorial yang diberikan, dan ketikkan kembali kode program yang disajikan pada komputer masing-masing. Referensi utama untuk kode-kode algoritma pengurutan pada bahasa Python dapat diakses pada tautan berikut: runestone.academy/ns/books/published/pythonds/SortSearch/Objectives.html

Masalah Pengurutan

- Masalah pengurutan akan memberikan suatu koleksi nilai yang belum terurut yang harus diurutkan oleh program.
- Mengurutkan data penting dalam pemrograman: mempercepat proses *searching* (pencarian) suatu nilai pada data.
- Contoh :
  - Saat mengakses aplikasi e-mail, kita mengurutkan email berdasarkan waktu yang terkini hingga yang paling lampau.
  - Saat mencari barang di toko online, kita dapat mengurutkan barang berdasarkan berbagai kriteria, misalnya harga terendah, harga tertinggi, atau reputasi.
- Hal-hal yang perlu diperhatian pada suatu masalah pengurutan:
  - Nilai yang akan diurutkan.
  - Kriteria pengurutan.
  - Banyaknya nilai yang akan diurutkan.

Algoritma Pengurutan

- Algoritma pengurutan akan terdiri atas dua elemen, yaitu adanya suatu himpunan nilai yang akan diurutkan dan kriteria pengurutan.
- Himpunan nilai yang diurutkan dapat berupa apapun, seperti angka dan string.
- Kriteria pengurutan sendiri merupakan aturan yang digunakan untuk menyatakan bahwa suatu nilai lebih kecil atau lebih besar daripada nilai yang

lain. Umumnya, kriteria pengurutan terbagi atas *ascending* (terurut menaik) dan *descending* (terurut menurun).

- Untuk mengurutkan objek yang memiliki banyak atribut, misalnya titik-titik yang terdapat pada koordinat tiga dimensi (x, y, z), kriteria pengurutan perlu didefiniskan di dalam program.
- Selain itu, pengurutan bisa saja dilakukan berdasarkan dua atribut, misalnya barang pada toko online diurutkan berdasarkan harga terendah. Jika ada dua barang yang harganya sama, maka barang dengan reputasi tertinggi akan disajikan sebelumnya.
- Terdapat banyak algoritma pengurutan, seperti *bubble sort*, *insertion sort*, *selection sort*, *quick sort*, *merge sort*, *counting sort*, *radix sort*, dan lain sebagainya. Masing-masing memiliki karakteristik tersendiri.
- Beberapa kondisi yang perlu diperhatikan saat mengurutkan:
  - Koleksi sudah terurut.
  - Koleksi terurut, tapi secara terbalik.
  - Koleksi tidak terurut.

Tabel 1. Perbandingan kompleksitas waktu (kecepatan) dari algoritma pengurutan yang dipelajari pada modul ini.

Algoritma	**Kompleksitas Waktu**		
	Terbaik	**Rata-Rata**	**Terburuk**
Selection Sort	Ω(n^2)	θ(n^2)	O(n^2)
Bubble Sort	Ω(n)	θ(n^2)	O(n^2)
Insertion Sort	Ω(n)	θ(n^2)	O(n^2)
Shell Sort	Ω(n log(n))	θ(n log(n))	O(n^2)
Merge Sort	Ω(n log(n))	θ(n log(n))	O(n log(n))
Quick Sort	Ω(n log(n))	θ(n log(n))	O(n^2)
Tim Sort	Ω(n)	θ(n log(n))	O(n log (n))

Sumber: geeksforgeeks.org/time-complexities-of-all-sorting-algorithms

Pengurutan pada Python

Bahasa pemrograman lazimnya telah memiliki fungsi standar untuk melakukan pengurutan yang dapat digunakan oleh pemrogram. Pada Python, pengurutan dilakukan menggunakan fungsi `sorted()` yang di dalammnya bekerja dengan menggunakan algoritma pengurutan Tim sort. Selain itu, struktur data seperti list telah memiliki fungsi `sort()` sebagai salah satu fungsi anggotanya. Perhatikan dan coba Program 1 dan Program 2 berikut.

Program 1. Fungsi `sorted()` pada Python untuk mengurutkan list.

```
a = [1, 3, 5, 7, 9, 0, 2, 4, 6, 8]
print(sorted(a))
print(a)

# Output:
# [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
# [1, 3, 5, 7, 9, 0, 2, 4, 6, 8]
```

Program 2. Fungsi `sort()` pada Python untuk mengurutkan list.

```
a = [1, 3, 5, 7, 9, 0, 2, 4, 6, 8]

a.sort()
print(a)

# Output:
# [1, 3, 5, 7, 9, 0, 2, 4, 6, 8]
```

Diskusikan:

Apa perbedaan penggunaan fungsi sorted() dengan sort() pada Python tersebut? Seperti apa kondisi list setelah kedua fungsi tersebut dieksekusi?

Implementasi Algoritma *Bubble Sort*

Prinsip *bubble sort* dalam mengurutkan data sebanyak N sederhana, yaitu program akan menelusuri list sebanyak N-1 kali (pada Program 3, nilai ini disebut sebagai passnum), masing-masing dari elemen ke-0 hingga elemen ke N-2. Setiap elemen akan dibandingkan dengan elemen selanjutnya. Apabila elemen tersebut posisinya tertukar, maka kedua elemen tersebut akan ditukar. Implementasi dari algoritma ini dapat dilihat pada Program 3.

Program 3. Implementasi algoritma *bubble sort* pada Python.

```
def bubbleSort(data):
  for passnum in range(len(data)-1,0,-1):
    for i in range(passnum):
      if data[i]>data[i+1]:
        # Tukar dua data bersebelahan yang urutannya salah
        temp = data[i]
        data[i] = data[i+1]
        data[i+1] = temp

data = [54,26,93,17,77,31,44,55,20]
bubbleSort(data)
print(data)

# Output
# [17, 20, 26, 31, 44, 54, 55, 77, 93]
```

Kelemahan dari Program 3 adalah program akan terus menelusuri dan membandingkan elemen list walaupun list tersebut telah terurut. Agar program berhenti ketika list telah terurut, kita dapat menambahkan suatu *flag* `exchanges` pada program seperti para Program 4. *Flag* ini akan bernilai *true* apabila pada iterasi sebelumnya terdapat setidaknya satu kali penukaran. Apabila pada iterasi sebelumnya sama sekali tidak terdapat pertukaran, *flag* akan bernilai *false* dan program pun akan berhenti dengan sendirinya.

Program 4. Implementasi algoritma *bubble sort* yang lebih efisien.

```
def shortBubbleSort(alist):
    exchanges = True
```

```
    passnum = len(alist)-1
    while passnum > 0 and exchanges:
       exchanges = False
       for i in range(passnum):
           if alist[i]>alist[i+1]:
               exchanges = True
               temp = alist[i]
               alist[i] = alist[i+1]
               alist[i+1] = temp
       passnum = passnum-1

alist=[20,30,40,90,50,60,70,80,100,110]
shortBubbleSort(alist)
print(alist)
```

Latihan:
Ketik kembali kode program di atas. Setelah itu, modifikasilah program yang awalnya mengurutkan secara menaik (*ascending*) menjadi mengurutkan secara menurun (*descending*).

Implementasi Algoritma *Selection Sort*

Prinsip dasar dari *selection sort* adalah program akan mencari nilai terbesar ke-*i*, dan akan meletakkannya pada posisi yang sesuai pada *list*. Misalnya, pada iterasi awal program akan mencari nilai yang paling besar dan menukarnya dengan nilai yang berada pada posisi terakhir. Demikian seterusnya hingga seluruh *list* terurut. Pada Program 5, dapat terlihat bahwa pada setiap iterasi, program akan mencari indeks tempat elemen terbesar tersebut berada yang disimpan pada variabel `positionOfMax`.

Program 5. Implementasi algoritma *selection sort*.

```
def selectionSort(data):
```

```
    for fillslot in range(len(data)-1,0,-1):
        positionOfMax=0
        for location in range(1,fillslot+1):
            if data[location]>data[positionOfMax]:
                positionOfMax = location

        # Swap
        temp = data[fillslot]
        data[fillslot] = data[positionOfMax]
        data[positionOfMax] = temp

data = [54,26,93,17,77,31,44,55,20]
selectionSort(data)
print(data)
```

Latihan:
Ketik kembali kode program di atas. Setelah itu, modifikasilah program yang awalnya mengurutkan secara menaik (*ascending*) menjadi mengurutkan secara menurun (*descending*).

Implementasi Algoritma *Insertion Sort*

Insertion sort merupakan sebuah teknik pengurutan dengan cara membandingkan dan mengurutkan dua data pertama pada *list*, kemudian membandingkan data para *list* berikutnya apakah sudah berada di tempat semestinya. Algoritma *insertion sort* seperti proses pengurutan kartu yang berada di tangan kita. Algoritma ini dapat mengurutkan data dari besar ke kecil (*ascending*) dan kecil ke besar (*descending*). Proses ini disajikan pada Program 6.

Program 6. Implementasi algoritma *insertion sort*.

```
def insertionSort(data):
    for index in range(1,len(data)):

      currentvalue = data[index]
      position = index

      while position>0 and data[position-1]>currentvalue:
          data[position]=data[position-1]
          position = position-1
```

```
    data[position]=currentvalue

data = [54,26,93,17,77,31,44,55,20]
insertionSort(data)
print(data)
```

Latihan:
Ketik kembali kode program di atas. Setelah itu, modifikasilah program yang awalnya mengurutkan secara menaik (*ascending*) menjadi mengurutkan secara menurun (*descending*).

Implementasi Algoritma *Shell Sort*

Salah satu kelemahan *insertion sort* adalah operasi pergeseran yang besar. Untuk mengurangi jumlah pergeseran tersebut, ide yang muncul adalah dengan memperbesar jarak (*gap*) pergeseran yang dilakukan, sehingga nilai yang posisinya jauh dari tempat seharusnya dapat dipindahkan ke posisi yang tepat dengan jumlah pergeseran yang lebih kecil. Ide tersebut melahirkan *shell sort*.

Shell sort adalah algoritma pengurutan (*sorting*) yang membagi *list* menjadi beberapa *sublist* yang lebih kecil yang kemudian diurutkan secara terpisah menggunakan algoritma pengurutan sederhana seperti *insertion sort*. Kemudian, *sublist* tersebut digabungkan kembali dengan ukuran yang lebih besar dan diurutkan kembali menggunakan algoritma pengurutan sederhana tersebut. Shell sort pertama kali dikembangkan oleh Donald Shell pada tahun 1959, dan sering kali dianggap sebagai perbaikan dari algoritma *insertion sort*. Salah satu kelebihan dari shell sort adalah ia dapat melakukan pengurutan dengan lebih cepat daripada insertion sort pada daftar data yang ukurannya cukup besar.

Fungsi `gapInsertionSort` merupakan implementasi dari algoritma *insertion sort* dengan modifikasi pada nilai increment atau gap yang digunakan. Pada *insertion sort*, nilai *increment* atau *gap* yang digunakan adalah 1, yaitu mengurutkan data secara satu per satu dari awal sampai akhir data. Namun pada *shell sort*, nilai

increment atau *gap* yang digunakan lebih besar dari 1 dan kemudian diubah-ubah pada setiap iterasi, sehingga data dapat diurutkan dengan lebih efisien.

Fungsi `gapInsertionSort` menerima tiga parameter, yaitu `data` yang merupakan list yang akan diurutkan, `start` yang merupakan indeks awal dari *sublist* yang akan diurutkan, dan *gap* yang merupakan jarak antara dua elemen yang akan dibandingkan pada saat pengurutan. Fungsi ini akan membandingkan setiap elemen dalam *sublist* dengan elemen yang memiliki jarak *gap* pada *sublist* tersebut. Jika nilai elemen yang memiliki jarak *gap* lebih besar dari nilai elemen yang sedang dibandingkan, nilai elemen tersebut akan dipindahkan ke posisi yang lebih besar dalam *sublist*.

Fungsi `gapInsertionSort` dipanggil pada bagian utama dari fungsi `shellSort` pada setiap iterasi saat nilai `sublistcount` lebih besar dari 0. Pada setiap iterasi, fungsi `gapInsertionSort` akan dipanggil untuk setiap *sublist* dalam data dengan indeks awal yang berbeda, dan *gap* yang berbeda pula, sehingga *sublist* dapat diurutkan secara terpisah. Setelah itu, nilai `sublistcount` akan dibagi dua dan diulang lagi hingga nilai `sublistcount` sama dengan 0. Pada akhirnya, data akan terurut dengan algoritma *shell sort*.

Program 7. Implementasi algoritma *shell sort*.

```
def shellSort(data):
    sublistcount = len(data)//2
    while sublistcount > 0:

      for startposition in range(sublistcount):
        gapInsertionSort(data,startposition,sublistcount)

      print("After increments of size",sublistcount,
                                  "The list is",data)

      sublistcount = sublistcount // 2

def gapInsertionSort(data,start,gap):
    for i in range(start+gap,len(data),gap):

        currentvalue = data[i]
        position = i
```

```
        while position>=gap and data[position-gap]>currentvalue:
            data[position]=data[position-gap]
            position = position-gap

        data[position]=currentvalue

data = [54,26,93,17,77,31,44,55,20]
shellSort(data)
print(data)
```

Latihan:
Ketik kembali kode program di atas. Setelah itu, modifikasilah program yang awalnya mengurutkan secara menaik (*ascending*) menjadi mengurutkan secara menurun (*descending*).

Implementasi Algoritma *Merge Sort*

Merge sort adalah algoritma pengurutan (*sorting*) yang menggunakan pendekatan *divide-and-conquer*, yaitu dengan membagi daftar data (*list*) menjadi dua bagian yang lebih kecil, mengurutkan setiap bagian secara terpisah, dan kemudian menggabungkan dua bagian yang telah diurutkan menjadi satu daftar data yang terurut.

Algoritma merge sort terdiri dari dua tahap utama:

- **Pembagian (Divide)**: Pada tahap ini, daftar data dibagi menjadi dua bagian yang sama besar atau hampir sama besar. Jika jumlah elemen pada daftar data ganjil, maka salah satu bagian akan memiliki elemen lebih banyak daripada yang lain.
- **Penggabungan (Conquer)**: Pada tahap ini, dua bagian yang telah diurutkan digabungkan menjadi satu daftar data yang terurut. Setiap elemen dari dua bagian yang sedang digabungkan akan dibandingkan dan ditempatkan pada posisi yang sesuai dengan urutan pengurutan.

Proses penggabungan pada *merge sort* dilakukan dengan cara membuat sebuah daftar data baru yang kosong, kemudian membandingkan elemen pertama dari masing-masing bagian dan menempatkannya pada daftar baru dengan urutan yang benar. Kemudian, elemen kedua dari setiap bagian dibandingkan lagi dan ditempatkan pada daftar baru dengan urutan yang benar, dan begitu seterusnya hingga semua elemen pada daftar baru terisi dan terurut.

Kelebihan dari algoritma *merge sort* adalah ia dapat melakukan pengurutan dengan cepat pada daftar data yang sangat besar, dan dapat diimplementasikan dengan mudah menggunakan rekursi. Namun, kelemahan dari *merge sort* adalah ia memerlukan penggunaan memori tambahan untuk menyimpan daftar data baru yang terurut, sehingga dapat memakan banyak ruang pada memori.

Program 8. Implementasi algoritma *merge sort*.

```
def mergeSort(data):
  print("Splitting ",data)
  if len(data)>1:
    mid = len(data)//2
    lefthalf = data[:mid]
    righthalf = data[mid:]

    mergeSort(lefthalf)
    mergeSort(righthalf)

    i=0
    j=0
    k=0
    while i < len(lefthalf) and j < len(righthalf):
      if lefthalf[i] <= righthalf[j]:
        data[k]=lefthalf[i]
        i=i+1
      else:
        data[k]=righthalf[j]
        j=j+1
      k=k+1

    while i < len(lefthalf):
      data[k]=lefthalf[i]
      i=i+1
      k=k+1
```

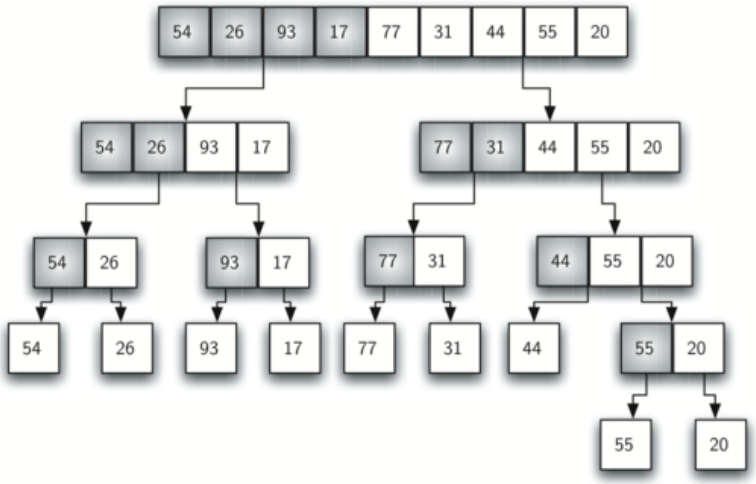
```
    while j < len(righthalf):
      data[k]=righthalf[j]
      j=j+1
      k=k+1
  print("Merging ",data)

data = [54,26,93,17,77,31,44,55,20]
mergeSort(data)
print(data)
```

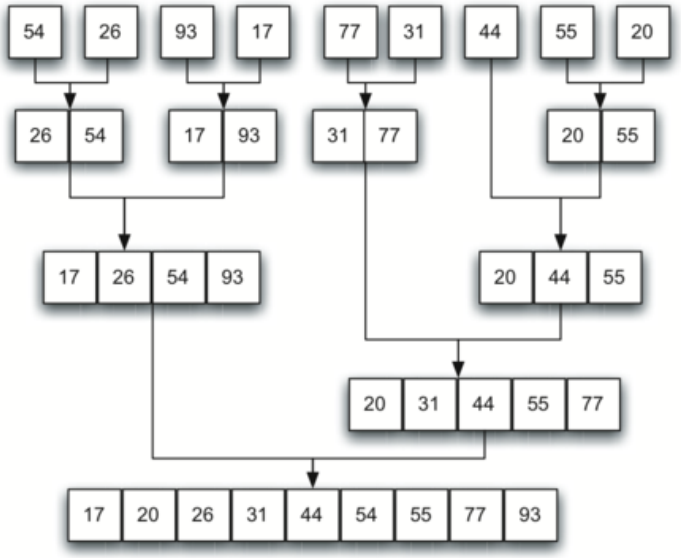
Latihan:

Ketik kembali kode program di atas. Setelah itu, modifikasilah program yang awalnya mengurutkan secara menaik (*ascending*) menjadi mengurutkan secara menurun (*descending*).

Perhatikan *output* pada program *merge sort* di atas, dan bandingkan dengan representasi pohon berikut:



Proses Divide



Proses Conquer

Implementasi Algoritma Quick Sort

Quick sort adalah algoritma pengurutan (*sorting*) yang menggunakan pendekatan divide-and-conquer, yaitu dengan membagi daftar data (array) menjadi dua bagian yang lebih kecil, dan kemudian mengurutkan setiap bagian secara terpisah. Algoritma ini didasarkan pada teknik pemilihan elemen pivot, yaitu elemen yang dipilih sebagai acuan dalam membagi daftar data menjadi dua bagian. Setelah pemilihan elemen pivot, elemen-elemen pada daftar data yang lebih kecil dari pivot ditempatkan pada sisi kiri pivot, sedangkan elemen-elemen yang lebih besar ditempatkan pada sisi kanan pivot.

Setelah pemisahan elemen, kedua bagian akan diurutkan secara terpisah menggunakan algoritma quick sort, dengan melakukan pemilihan elemen pivot lagi pada setiap bagian dan membaginya lagi menjadi dua bagian yang lebih kecil. Proses ini diulang-ulang hingga seluruh elemen pada daftar data terurut.

Kelebihan dari algoritma quick sort adalah kecepatan pengurutan yang relatif tinggi dan efisien pada daftar data yang besar. Algoritma ini juga dapat diimplementasikan secara mudah dengan menggunakan rekursi dan tidak

memerlukan memori tambahan untuk penyimpanan hasil pengurutan. Namun, kelemahan dari quick sort adalah bahwa waktu pengurutan tergantung pada pemilihan elemen pivot yang tepat, sehingga pemilihan yang buruk dapat mempengaruhi kinerja algoritma. Quick sort juga tidak stabil, yaitu tidak mempertahankan urutan relatif dari elemen-elemen yang sama pada daftar data.

Program 9. Implementasi algoritma *quick sort*.

```
def quickSort(data):
 quickSortHelper(data,0,len(data)-1)

def quickSortHelper(data,first,last):
 if first<last:
   splitpoint = partition(data,first,last)
   quickSortHelper(data,first,splitpoint-1)
   quickSortHelper(data,splitpoint+1,last)


def partition(data,first,last):
 pivotvalue = data[first]

 leftmark = first+1
 rightmark = last

 done = False
 while not done:

   while leftmark <= rightmark and data[leftmark] <= pivotvalue:
     leftmark = leftmark + 1

   while data[rightmark] >= pivotvalue and rightmark >= leftmark:
     rightmark = rightmark -1

   if rightmark < leftmark:
     done = True
   else:
     temp = data[leftmark]
     data[leftmark] = data[rightmark]
     data[rightmark] = temp

 temp = data[first]
 data[first] = data[rightmark]
 data[rightmark] = temp
```

```
 return rightmark

data = [54,26,93,17,77,31,44,55,20]
quickSort(data)
print(data)
```

Latihan:
Ketik kembali kode program di atas. Setelah itu, modifikasilah program yang awalnya mengurutkan secara menaik (*ascending*) menjadi mengurutkan secara menurun (*descending*).

Latihan Soal Pengurutan

Gunakanlah program-program pengurutan di atas untuk menyelesaikan permasalahan berikut:

Pak Budi adalah seorang manager sumber daya manusia di suatu perusahaan. Ia saat ini harus menseleksi pelamar kerja berdasarkan skor tes potensi akademik mereka. Skor tersebut disajikan dalam bentuk *list* dengan rentang nilai 0 - 100. Berikut adalah data hasil tes potensi akademik yang tersedia:

[43, 76, 12, 89, 33, 57, 98, 22, 68, 9]

Soal:
1. Jika Pak Budi akan meloloskan lima kandidat dengan nilai tertinggi, tuliskanlah skor lima kandidat tersebut dari yang paling tinggi hingga terendah.
2. Kandidat berapa saja yang lolos?

Topik 9: Pencarian (Searching)

Penulis: Auzi Asfarian, S.Komp., M.Kom

Aktivitas:
Ikutilah tutorial yang diberikan, dan ketikkan kembali kode program yang disajikan pada komputer masing-masing. Referensi utama untuk kode-kode algoritma pengurutan pada bahasa Python dapat diakses pada tautan berikut: runestone.academy/ns/books/published/pythonds/SortSearch/Objectives.html

Masalah Pencarian

- Masalah pencarian melibatkan suatu nilai yang dicari dan suatu koleksi atau struktur data.
- Masalah pencarian dilakukan untuk menemukan nilai tersebut di dalam koleksi. Terdapat beberapa variasi bentuk pencarian seperti berikut:
  - Menentukan apakah suatu nilai x ada di dalam koleksi. Nilai kembalian adalah true atau false.
  - Menentukan posisi nilai x pertama kali di dalam koleksi.
  - Menentukan seluruh posisi nilai x di dalam koleksi (jika suatu nilai bisa muncul lebih dari satu kali di dalam koleksi).

Algoritma Pencarian

- Suatu algoritma pencarian akan mengecek suatu koleksi data secara sistematik hingga akhirnya pertanyaan pencarian dapat ditemukan.
- Beberapa kondisi yang perlu diperhatikan:
  - Nilai yang dicari tidak ada di dalam koleksi.
  - Nilai yang dicari ada di dalam koleksi (satu kemunculan)
  - Nilali yang dicari ada di dalam koleksi (banyak kemunculan)

Pencarian pada Struktur Data di Python

Pada Python, terdapat dua cara untuk melakukan pencarian pada struktur data. Yang pertama adalah dengan menggunakan operator keanggotaan `in` dan `not in` seperti pada Program 1.

Program 1. Pencarian pada suatu list menggunakan operator keanggotaan in dan not in pada Python.

```
2 in [1, 3, 5, 7, 9]
4 in [0, 2, 4, 6, 8]
6 not in [1, 3, 5, 7, 9]
8 not in [0, 2, 4, 6, 8]

# Output:
# False
# True
# True
# False
```

Cara kedua adalah dengan menggunakan fungsi `index()` yang terdapat pada koleksi, misalnya pada *list*. Fungsi index() akan menerima tiga buah parameter, yaitu

`list_name.index(value, start, stop).`

Value adalah nilai yang dicari, sedangkan *start* dan *stop* adalah parameter opsional untuk membatasi pencarian pada elemen list antara indeks start dan indeks stop. Keluaran dari fungsi `index()` adalah alamat pertama *value* pada koleksi. Akan tetapi, apabila *value* tidak ada pada koleksi, fungsi `index()` akan mengalami *runtime error* seperti terlihat pada Program 2.

Program 2. Pencarian pada suatu list menggunakan fungsi index() pada koleksi.

```
buah = ['Apel', 'Buah Naga', 'Ceri']
x = buah.index("Ceri")
y = buah.index("Kedondong")
print(x)
print(y)

# Output:
# ValueError: 'Kedondong' is not in list
```

Untuk mengatasi *runtime error* tersebut, penggunaan fungsi `index()` harus disertai dengan block `try/except` seperti pada Program 3.

Program 3. Pencarian pada suatu list menggunakan fungsi index() pada koleksi.

```
buah = ['Apel', 'Buah Naga', 'Ceri']

try:
    x = buah.index("Ceri")
except ValueError:
    x = -1

try:
    y = buah.index("Kedondong")
except ValueError:
    y = -1

print(x)
print(y)

# Output
# 2
# -1
```

Latihan:

Diberikan suatu list berikut:

data = [1, 3, 5, 7, 9, 0, 2, 4, 6, 8]

Apakah keluaran dari:

- data.index(3, 0, 3)
- data.index(3, 5, 6)
- data.index(3, 3, 0)
- data.index(3, 0, 11)

Implementasi Algoritma Sequential Search

Ide dari *sequential search* sangat sederhana: kunjungi seluruh elemen dari awal hingga akhir pada koleksi hingga nilai yang dicari ditemukan atau seluruh elemen telah dicek seperti pada Program 4. Cara ini tidak efisien karena operasi pengecekan terburuk sama dengan jumlah data yang tersedia, O(*n*). Akan tetapi, inilah cara terbaik yang dapat dilakukan untuk melakukan pencarian pada suatu koleksi yang tidak terurut.

Program 4. Implementasi algoritma *sequential search* pada Python

```
def sequentialSearch (data, value):
  pos = 0
  found = False
  while pos <len (data) and not found:
    if data[pos] == value:
       found = True
    else:
       pos = pos + 1
  return found

data = [1, 2, 32, 8, 17, 19, 42, 13, 0, 2]
print (sequentialSearch (data, 3))
print (sequentialSearch (data, 13))
```

Latihan:

Modifikasi Program 4 di atas hingga program tersebut:

1. Mengembalikan posisi (indeks) pertama dari nilai yang dicari pada list. Apabila nilai tidak ditemukan, kembalikan -1.
2. Mengembalikan seluruh posisi dari nilai yang dicari pada *list* (Contoh: pada saat mencari 2, nilai yang dikembalikan adalah [1, 9].

Implementasi Algoritma Binary Search

Berbeda dengan *sequential search*, pada binary search, koleksi harus terurut terlebih dahulu. Dengan kondisi koleksi yang telah terurut ini, pencarian dapat

dilakukan lebih cepat dengan mengecek nilai tengah dari suatu koleksi. Ketika kita membandingkan nilai yang dicari dengan nilai tengah, kita akan memperoleh tepat satu dari tiga informasi berikut:

- Nilai yang dicari ternyata adalah nilai tengah.
- Nilai yang dicari kurang dari nilai tengah.
- Nilai yang dicari lebih dari nilai tengah.

Dengan kata lain, begitu kita membandingkan, kita dapat mengeliminasi setengah dari kemungkinan lokasi munculnya nilai yang dicari. Sebagai contoh:

Data = [0, 1, 2, 3, 4, 5, 6, 7, 8]

Ketika kita mencari nilai 3 pada data, perbandingan pertama akan membandingkan 3 dengan nilai paling tengah yaitu 4. Karena 3 < 4, kita bisa mengeliminasi pencarian pada indeks ke-5 hingga ke-8 karena pasti seluruh nilai pada sisi tersebut lebih besar daripada nilai tengah dan nilai yang dicari. Selanjutnya kita hanya perlu membandingkan sisi kiri daripada *list* di atas dengan cara yang serupa. Algoritma inilah yang disebut sebagai *binary search*. Implementasi binary search pada Python dapat dilihat pada Program 5.

Program 5. Implementasi algoritma *binary search* pada Python

```
def binarySearch(data, item):
    first = 0
    last = len(data)-1
    found = False

    while first<=last and not found:
        midpoint = (first + last)//2
        if data[midpoint] == item:
            found = True
        else:
            if item < data[midpoint]:
                last = midpoint-1
            else:
                first = midpoint+1
    return found
```

```
data = [1, 2, 32, 8, 17, 19, 42, 13, 0, 2]
print(binarySearch(data, 3))
print(binarySearch(data, 13))
```

Karena pada setiap perbandingan *binary search* akan mengeliminasi $n/2$ kemungkinan lokasi, maka jumlah perbandingan maksimal yang dilakukan akan sama dengan $^{2}\log n$. Dengan kata lain, jika ukuran koleksi adalah 1000 misalnya, maka *binary search* hanya memerlukan sebanyak $^{2}\log 1000$ atau sebanyak 10 kali perbandingan, berbeda jauh dengan sequential search yang memerlukan maksimal hingga 1000 kali pencarian.

Latihan:

Modifikasi Program 4 di atas hingga program tersebut:

1. Mengembalikan posisi (indeks) pertama dari nilai yang dicari pada list. Apabila nilai tidak ditemukan, kembalikan -1.
2. Mengembalikan seluruh posisi dari nilai yang dicari pada *list* (Contoh: pada saat mencari 2, nilai yang dikembalikan adalah [1, 9].

Latihan Soal Pencarian

Pak Raharjo sedang mengolah data nilai tes potensi akademik pelamar kerja di kantornya. Datanya adalah sebagai berikut:

[43, 76, 12, 89, 33, 57, 98, 22, 68, 9]

Untuk kepentingan seleksi berkas, Pak Raharjo perlu mendapatkan informasi-informasi sebagai berikut:

- Siapa saja yang memiliki nilai sama dengan X.
- Siapa saja yang memiliki nilai lebih besar daripada X.

Untuk masing-masing poin di atas, buatlah program untuk membantu Pak Raharjo. Buatlah dua versi program, yang satu menggunakan *sequential search* dan satu lagi menggunakan *binary search*. Gunakanlah algoritma pengurutan

---

untuk membantu Anda mempersiapkan data tersebut sehingga dapat digunakan oleh *binary search*.